

Entorno personal de aprendizaje:

CREACIÓN DE PÁGINAS WEB



Información SEDE 1



Ministerio de
**TECNOLOGÍAS
DE LA INFORMACIÓN
Y COMUNICACIÓN**

Dirección: Complejo Santos E2 - Gral. Santos 1170 c/ Concordia
Teléfono: (+595 21) 217 9000
Correo electrónico: comunicacion@mitic.gov.py Mesa de entrada: sgeneral@mitic.gov.py
Asunción – Paraguay

Curso de creación de páginas web: Unidad 6

Validación de Datos en un Formulario HTML.

Todas las imágenes presentes en este material de apoyo educativo han sido reconocidas y, en todos los casos, son de uso libre. Todas las fuentes bibliográficas utilizadas en este material de apoyo educativo han son citadas mediante el gestor de referencias bibliográficas Zotero, desarrollado por el Center for History and New Media de la Universidad George Mason, utilizando el estilo de la American Psychological Association Séptima Edición.



Ministerio de
**TECNOLOGÍAS
DE LA INFORMACIÓN
Y COMUNICACIÓN**

 **GOBIERNO
NACIONAL**

*Paraguay
de la gente*

Contenido

1. Validación de Datos en un Formulario HTML.	4
2. HTML	4
2.1. Explicación del código HTML.....	5
3. CSS (style.css)	8
3.1. Explicación del código CSS	9
4. JavaScript (script.js).....	12
4.1. Explicación del código JavaScript	13
Referencias bibliográficas	16

1. Validación de Datos en un Formulario HTML.

Este ejemplo incluye un formulario básico con campos de nombre, correo electrónico y un mensaje, y muestra cómo realizar la validación de datos antes de enviar el formulario.

2. HTML

Un formulario con campos para el nombre, el correo electrónico y un mensaje.

Un elemento `<p>` para mostrar mensajes de error.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width,
initial-scale=1.0">
  <title>Formulario con Validación</title>
  <link rel="stylesheet" href="style.css">
  <script defer src="script.js"></script>
</head>
<body>
  <div class="container">
    <h1>Formulario de Contacto</h1>
    <form id="contactForm">
      <label for="name">Nombre:</label>
      <input type="text" id="name" name="name"
required>

      <label for="email">Correo Electrónico:</label>
      <input type="email" id="email" name="email"
required>

      <label for="message">Mensaje:</label>
      <textarea id="message" name="message"
required></textarea>
      <input type="submit" value="Enviar">
    </form>
    <p id="errorMessage" style="color: red;"></p>
  </div>
</body>
</html>
```

2.1. Explicación del código HTML

- Declaración del tipo de documento y apertura de la etiqueta HTML

```
<!DOCTYPE html>  
<html lang="en">
```

<!DOCTYPE html>: Declara el tipo de documento y la versión del HTML que se está utilizando. En este caso, es HTML5.

<html lang="en">: Indica el inicio del documento HTML y establece el idioma del contenido como inglés.

- Sección <head>

```
<head>  
  <meta charset="UTF-8">  
  <meta name="viewport" content="width=device-width,  
initial-scale=1.0">  
  <title>Formulario con Validación</title>  
  <link rel="stylesheet" href="style.css">  
  <script defer src="script.js"></script>  
</head>
```

<meta charset="UTF-8">: Especifica la codificación de caracteres para el documento, en este caso UTF-8, que es una codificación estándar que soporta todos los caracteres y símbolos comunes.

<meta name="viewport" content="width=device-width, initial-scale=1.0">: Esta metaetiqueta es crucial para asegurar que el sitio web se vea bien en todos los dispositivos, ajustando el ancho del contenido a la anchura del dispositivo.

<title>Formulario con Validación</title>: Define el título del documento que se muestra en la pestaña del navegador.

<link rel="stylesheet" href="style.css">: Vincula el documento HTML con una hoja de estilos externa (style.css). Esto se utiliza para aplicar

estilos CSS al documento.

<script defer src="script.js"></script>: Vincula un archivo JavaScript externo (script.js). El atributo defer asegura que el script se ejecute después de que el HTML se haya cargado completamente.

- Sección <body>

```
<body>
  <div class="container">
    <h1>Formulario de Contacto</h1>
    <form id="contactForm">
      <label for="name">Nombre:</label>
      <input type="text" id="name" name="name"
required>

      <label for="email">Correo Electrónico:</label>
      <input type="email" id="email" name="email"
required>

      <label for="message">Mensaje:</label>
      <textarea id="message" name="message"
required></textarea>

      <input type="submit" value="Enviar">
    </form>
    <p id="errorMessage" style="color: red;"></p>
  </div>
</body>
<body>: Contiene todo el contenido visible del documento web.
```

<div class="container">: Define un contenedor para el contenido del formulario. La clase container se utiliza para aplicar estilos CSS específicos.

<h1>Formulario de Contacto</h1>: Un encabezado de nivel 1 que titula el formulario.

<form id="contactForm">: Define un formulario y le asigna un identificador único (id="contactForm") que se utilizará en JavaScript para manipular y validar el formulario.

- Campos del formulario

```
<label for="name">Nombre:</label>
<input type="text" id="name" name="name" required>

<label for="email">Correo Electrónico:</label>
<input type="email" id="email" name="email" required>

<label for="message">Mensaje:</label>
<textarea id="message" name="message" required></textarea>

<input type="submit" value="Enviar">
```

<label for="name">Nombre:</label>: Etiqueta que describe el campo de entrada que sigue. El atributo for debe coincidir con el id del campo de entrada asociado.

<input type="text" id="name" name="name" required>: Campo de entrada de texto para el nombre. El atributo required indica que este campo es obligatorio.

<label for="email">Correo Electrónico:</label>: Etiqueta para el campo de correo electrónico.

<input type="email" id="email" name="email" required>: Campo de entrada de correo electrónico. El tipo email asegura que el navegador valide el formato del correo.

<label for="message">Mensaje:</label>: Etiqueta para el área de texto del mensaje.

<textarea id="message" name="message" required></textarea>: Área de texto para el mensaje. También es obligatorio.

<input type="submit" value="Enviar">: Botón para enviar el formulario.

- Elemento para mostrar mensajes de error

```
<p id="errorMessage" style="color: red;"></p>
```

`<p id="errorMessage" style="color: red;"></p>`: Un párrafo vacío que se utilizará para mostrar mensajes de error de validación. Se le asigna un identificador (`id="errorMessage"`) y se le aplica un estilo en línea para que el texto aparezca en rojo.

3. CSS (`style.css`)

```
body {
  font-family: Arial, sans-serif;
  background-color: #f0f0f0;
  margin: 0;
  padding: 0;
}
.container {
  width: 50%;
  margin: auto;
  background-color: #fff;
  padding: 20px;
  border-radius: 5px;
  box-shadow: 0 0 10px rgba(0, 0, 0, 0.1);
}
h1 {
  text-align: center;
}
form {
  display: flex;
  flex-direction: column;
}

label {
  margin-top: 10px;
}
input, textarea {
  padding: 10px;
  margin-top: 5px;
  border: 1px solid #ccc;
  border-radius: 3px;
}
input[type="submit"] {
  margin-top: 20px;
  background-color: #0066cc;
  color: #fff;
  border: none;
  cursor: pointer;
  border-radius: 3px;
}
```



```
input[type="submit"]:hover {  
  background-color: #004080;  
}
```

3.1. Explicación del código CSS

- Estilo para el body

```
body {  
  font-family: Arial, sans-serif;  
  background-color: #f0f0f0;  
  margin: 0;  
  padding: 0;  
}
```

font-family: Arial, sans-serif;: Define la fuente que se usará en todo el documento. Si Arial no está disponible, se usará una fuente sans-serif genérica.

background-color: #f0f0f0;: Establece el color de fondo del documento a un gris claro.

margin: 0; padding: 0;: Elimina los márgenes y el relleno predeterminados del navegador, asegurando que el contenido comience desde el borde de la ventana.

- Estilo para el contenedor .container

```
.container {  
  width: 50%;  
  margin: auto;  
  background-color: #fff;  
  padding: 20px;  
  border-radius: 5px;  
  box-shadow: 0 0 10px rgba(0, 0, 0, 0.1);  
}
```

width: 50%;: Establece el ancho del contenedor al 50% del ancho de la ventana del navegador.

margin: auto;: Centra el contenedor horizontalmente.

background-color: #fff;: Establece el fondo del contenedor a blanco.

padding: 20px;: Añade un relleno de 20 píxeles dentro del contenedor.

border-radius: 5px;: Redondea las esquinas del contenedor con un radio de 5 píxeles.

box-shadow: 0 0 10px rgba(0, 0, 0, 0.1);: Añade una sombra suave alrededor del contenedor para darle un efecto de elevación.

- Estilo para el encabezado h1

```
h1 {  
  text-align: center;  
}
```

text-align: center;: Centra el texto del encabezado h1.

- Estilo para el formulario form

```
form {  
  display: flex;  
  flex-direction: column;  
}
```

display: flex;: Define el formulario como un contenedor flexible.

flex-direction: column;: Alinea los elementos del formulario en una columna, uno debajo del otro.

- Estilo para las etiquetas label

```
label {  
  margin-top: 10px;  
}
```

margin-top: 10px;: Añade un margen superior de 10 píxeles a las etiquetas, creando espacio entre ellas y los elementos anteriores.

- Estilo para los campos de entrada input y el área de texto textarea

```
input, textarea {  
  padding: 10px;  
  margin-top: 5px;  
  border: 1px solid #ccc;  
  border-radius: 3px;  
}
```

padding: 10px;: Añade un relleno de 10 píxeles dentro de los campos de

entrada y el área de texto.

margin-top: 5px;: Añade un margen superior de 5 píxeles para separar los campos de entrada de las etiquetas.

border: 1px solid #ccc;: Añade un borde de 1 píxel de ancho y color gris claro (#ccc) alrededor de los campos.

border-radius: 3px;: Redondea las esquinas de los campos con un radio de 3 píxeles.

- Estilo para el botón de envío `input[type="submit"]`

```
input[type="submit"] {  
  margin-top: 20px;  
  background-color: #0066cc;  
  color: #fff;  
  border: none;  
  cursor: pointer;  
  border-radius: 3px;  
}
```

margin-top: 20px;: Añade un margen superior de 20 píxeles para separar el botón de los campos anteriores.

background-color: #0066cc;: Establece el color de fondo del botón a un azul (#0066cc).

color: #fff;: Establece el color del texto del botón a blanco.

border: none;: Elimina el borde alrededor del botón.

cursor: pointer;: Cambia el cursor a un puntero cuando se pasa sobre el botón, indicando que es interactivo.

border-radius: 3px;: Redondea las esquinas del botón con un radio de 3 píxeles.

- Estilo para el botón de envío al pasar el cursor `input[type="submit"]:hover`

```
input[type="submit"]:hover {
  background-color: #004080;
}
```

background-color: #004080; Cambia el color de fondo del botón a un azul más oscuro (#004080) cuando se pasa el cursor sobre él, proporcionando un efecto visual que indica que el botón es interactivo.

4. JavaScript (script.js)

```
document.addEventListener('DOMContentLoaded', function() {
  const form = document.getElementById('contactForm');
  const errorMessage =
document.getElementById('errorMessage');
  form.addEventListener('submit', function(event) {
    // Clear any previous error message
    errorMessage.textContent = '';

    // Validate form fields
    const name = form.name.value.trim();
    const email = form.email.value.trim();
    const message = form.message.value.trim();

    if (!name || !email || !message) {
      errorMessage.textContent = 'Todos los campos son
obligatorios.';
      event.preventDefault(); // Prevent form
submission
      return;
    }
    if (!validateEmail(email)) {
      errorMessage.textContent = 'Por favor, introduce
una dirección de correo electrónico válida.';
      event.preventDefault(); // Prevent form
submission
      return;
    }
    // Additional validations can be added here
    // If all validations pass, the form will be
submitted
  });
  function validateEmail(email) {
    const re = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
    return re.test(email);
  }
});
```

4.1. Explicación del código JavaScript

- Se añade un event listener para el evento submit del formulario:

```
form.addEventListener('submit', function(event) {
```

document.addEventListener('DOMContentLoaded', function() {...}): Este código asegura que el script se ejecute después de que el documento HTML haya sido completamente cargado. De esta manera, el código no intenta acceder a elementos del DOM que aún no existen.

form.addEventListener('submit', function(event) {...}): Aquí, se añade un "escuchador de eventos" (event listener) para el evento submit del formulario. Esto significa que cada vez que el usuario intente enviar el formulario, esta función será ejecutada.

- Se valida que todos los campos estén llenos:

```
const name = form.name.value.trim();
const email = form.email.value.trim();
const message = form.message.value.trim();

if (!name || !email || !message) {
  errorMessage.textContent = 'Todos los campos son obligatorios.';
  event.preventDefault(); // Prevent form submission
  return;
}
```

const name = form.name.value.trim();: Se obtiene el valor del campo "nombre" del formulario y se elimina cualquier espacio en blanco al principio y al final.

const email = form.email.value.trim();: Se obtiene el valor del campo "correo electrónico" y se elimina cualquier espacio en blanco.

const message = form.message.value.trim();: Se obtiene el valor del campo "mensaje" y se elimina cualquier espacio en blanco.

if (!name || !email || !message) {...}: Si alguno de los campos está vacío (es decir, después de eliminar los espacios en blanco, el valor es una cadena

vacía), se muestra un mensaje de error y se previene el envío del formulario usando `event.preventDefault()`.

- Se valida que el correo electrónico tenga un formato correcto:

```
if (!validateEmail(email)) {  
    errorMessage.textContent = 'Por favor, introduce una  
    dirección de correo electrónico válida.';  
    event.preventDefault(); // Prevent form submission  
    return;  
}
```

if (!validateEmail(email)) {...}: Se llama a la función `validateEmail` con el valor del correo electrónico. Si la función devuelve `false` (lo que significa que el correo electrónico no tiene un formato válido), se muestra un mensaje de error y se previene el envío del formulario.

Si alguna validación falla, se muestra un mensaje de error y se previene el envío del formulario:

errorMessage.textContent = '...': Esta línea de código actualiza el contenido de texto del elemento con el id `errorMessage` para mostrar un mensaje de error al usuario.

event.preventDefault(): Este método evita que el formulario se envíe. Sin esta línea, el formulario se enviaría normalmente (lo que recargaría la página), incluso si hay errores de validación.

- Función para validar el formato del correo electrónico:

```
function validateEmail(email) {  
    const re = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;  
    return re.test(email);  
}
```

const re = /^[^\s@]+@[^\s@]+\.[^\s@]+\$/; Este es un patrón (expresión regular) que describe un formato válido para una dirección de correo electrónico.

return re.test(email); El método `test` verifica si el valor del correo

electrónico cumple con el patrón de la expresión regular. Devuelve true si el correo electrónico es válido y false si no lo es.

Referencias bibliográficas

Equipo editorial de IONOS. (2022, 27 septiembre). *Resumen de los comandos Java más importantes*. IONOS Digital Guide. <https://www.ionos.com/es-us/digitalguide/paginas-web/desarrollo-web/comandos-java/>

Command en Java / Patrones de diseño. (s. f.). <https://refactoring.guru/es/design-patterns/command/java/example>

Plynko, P. (2024, 1 febrero). *Argumentos de la línea de comandos de Java*. CodeGym. <https://codegym.cc/es/groups/posts/es.664.argumentos-de-la-linea-de-comandos-de-java>

Java: cómo compilar y ejecutar desde línea de comandos | DAM. (s. f.). <https://ies-pablo-serrano.gitbook.io/dam/java-como-compilar-y-ejecutar-desde-linea-de-comandos>